

# ECE 315

Steven Knudsen, PhD, PEng

Lecture 27 :

- Using DMA to capture real world data



UNIVERSITY  
OF ALBERTA

# Last Lecture

- Pico VGA Example
- DMA + PIO Example

# This Lecture

- Using DMA to capture real world data

## Learning Objectives

- Explain why DMA is useful for efficiently capturing real-world analogue data in embedded systems; what are the benefits?
- Identify common real-world systems where analogue interfaces are present
- Describe the role of ADCs, DACs, and DMA within a digitally controlled analogue feedback system
- Understand how a round-robin sharing of a single ADC affects the effective sample rate
- Explain why the samples for the example needed to be de-interleaved



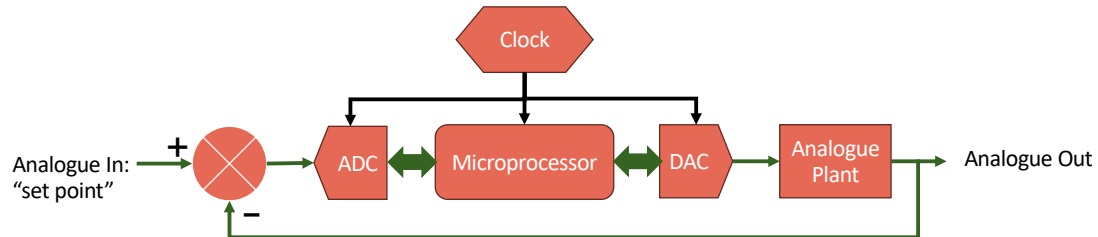
4

There are just some of the learning objectives that might come from this lecture.

## Using DMA to capture real world data

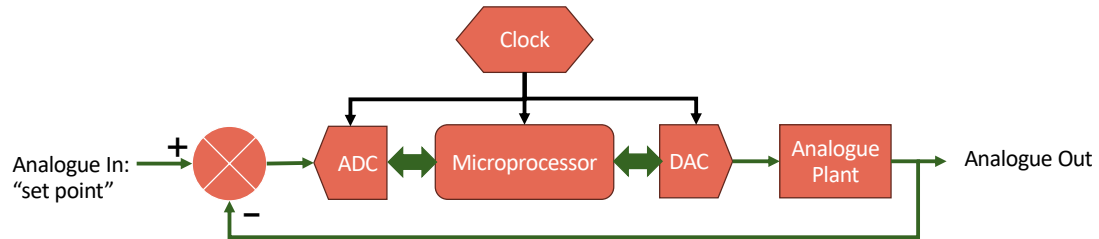
- It is very common to interface computers, microcontrollers to the real world
  - The Internet of Things (IoT), such as thermostats, cameras, switches, ...
  - Automotive systems,
  - Heating, ventilation, and air-conditioning (HVAC)
  - Building electrical and power systems
  - Biomedical systems
  - Human-interface systems
  - Audio entertainment systems
- DMA supports efficient data and signal gathering without CPU loading

## Example; Analogue Feedback Control



- A common configuration for modern control of analogue systems may take an analogue input that is a "set point"
  - Might be from a dial control, another system in the building, etc.
- More flexible to convert to a digital signal, process, then convert back to the desired output
- Microprocessor gives ability to update and dynamically control overall system

## Example; Analogue Feedback Control



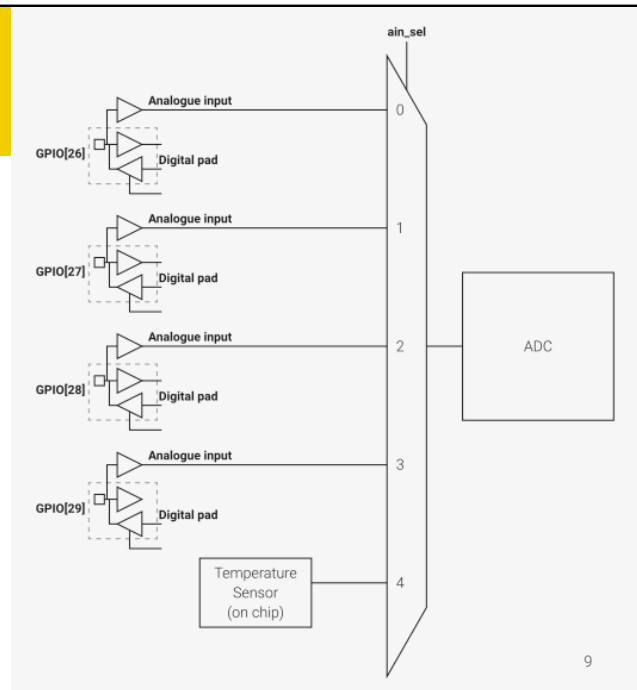
- On the microprocessor, DMA may be used to move input samples from the ADC into buffers for processing and move resulting data from output buffers to the DAC
- Allows us to use a less powerful CPU efficiently
  - Lower power consumption
  - Lower cost
  - Smaller footprint

## Example

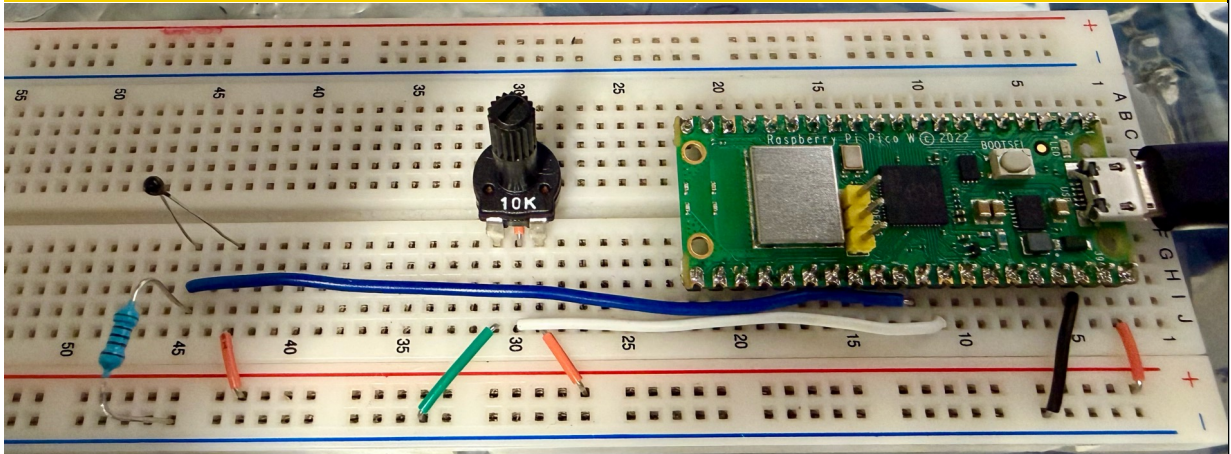
- Look at using DMA to completely offload the CPU
- The RP2040 has a built-in analogue-to-digital converter that is multiplexed to 5 channels
  - 1 channel is tied to an on-chip temperature sensor
- We will look at combining the multi-channel ADC and DMA to sample external and internal temperature, and a variable voltage
  - The voltage might represent the analogue input to a control system

## RP2040 ADC

- Connect thermistor to ADC via `ain_sel[0]`, `GPIO[26]`
- Connect potentiometer to ADC via `ain_sel[1]`, `GPIO[27]`
  - Acts as a voltage divider
- Connect temperature sensor to ADC via `ain_sel[4]`



## Pico + Thermistor + Potentiometer



## ece315\_dma\_adc\_x2.c

- All in one file program
- Start with includes and useful definitions
- Define the ring buffer type
  - Contains buffer, indices, count



```
/**
 * @file ece315_dma_adc_x2.c
 * @author : K. Steven Knudsen
 * @date : December 11, 2025
 *
 * @detail Samples three channels at 1 kSps:
 * - ADC0 (GPIO26) : Thermistor-resistor divider
 * - ADC1 (GPIO27) : Potentiometer variable voltage
 * - ADC4 (internal) : Internal temperature monitor
 *
 * Uses DMA with ping-pong buffering and ring buffer architecture
 */

#include <stdio.h>
#include <string.h>
#include "pico/stdlib.h"
#include "hardware/adc.h"
#include "hardware/dma.h"
#include "hardware/irq.h"

// Configuration
#define SAMPLE_RATE_HZ 3000 // 1 kSps total (1 kSps per channel)
#define RING_BUFFER_SIZE 1000 // Samples per channel
#define DMA_BUFFER_SIZE 768 // DMA transfer size (must be power of 2)
#define NUM_CHANNELS 3

// ADC channel definitions
#define ADC_CHANNEL_0 0 // GPIO26 - External input
#define ADC_CHANNEL_1 1 // GPIO27 - External input
#define ADC_CHANNEL_TEMP 4 // Internal temperature sensor

// Ring buffer structure
typedef struct {
    uint16_t buffer[RING_BUFFER_SIZE];
    volatile uint32_t write_idx;
    volatile uint32_t read_idx;
    volatile uint32_t count;
} ring_buffer_t;
```

11

## ece315\_dma\_adc\_x2.c cont'd

- Define all the global variables
- Define all ring buffer methods
- In C++ this would all go into a class
- Notice the use of modulo operator with the RING\_BUFFER\_SIZE
- Notice, too, that in `_put` and `_get` the state of the ring buffer is checked



```
// Global variables
static ring_buffer_t adc0_ring;
static ring_buffer_t adc1_ring;
static ring_buffer_t temp_ring;
static uint16_t dma_buffer_a[RING_BUFFER_SIZE];
static uint16_t dma_buffer_b[RING_BUFFER_SIZE];
static int dma_chan;
static volatile bool using_buffer_a = true;

// Ring buffer functions
static inline void ring_buffer_init(ring_buffer_t *rb) {
    rb->write_idx = 0;
    rb->read_idx = 0;
    rb->count = 0;
}

static inline bool ring_buffer_full(ring_buffer_t *rb) {
    return rb->count >= RING_BUFFER_SIZE;
}

static inline bool ring_buffer_empty(ring_buffer_t *rb) {
    return rb->count == 0;
}

static inline void ring_buffer_put(ring_buffer_t *rb, uint16_t value) {
    if (ring_buffer_full(rb)) {
        // Overwrite oldest sample
        rb->read_idx = (rb->read_idx + 1) % RING_BUFFER_SIZE;
    } else {
        rb->count++;
    }
    rb->buffer[rb->write_idx] = value;
    rb->write_idx = (rb->write_idx + 1) % RING_BUFFER_SIZE;
}

static inline uint16_t ring_buffer_get(ring_buffer_t *rb) {
    if (ring_buffer_empty(rb)) {
        return 0;
    }
    uint16_t value = rb->buffer[rb->read_idx];
    rb->read_idx = (rb->read_idx + 1) % RING_BUFFER_SIZE;
    rb->count--;
    return value;
}
```

12

## ece315\_dma\_adc\_x2.c cont'd

- Define all the global variables
- Define all ring buffer methods
- In C++ this would all go into a class
- Notice the use of modulo operator with the RING\_BUFFER\_SIZE
- Notice, too, that in `_put` and `_get` the state of the ring buffer is checked
- Should the `read_idx` and `write_idx` be checked as well?



```
// Global variables
static ring_buffer_t adc0_ring;
static ring_buffer_t adc1_ring;
static ring_buffer_t temp_ring;
static uint16_t dma_buffer_a[RING_BUFFER_SIZE];
static uint16_t dma_buffer_b[RING_BUFFER_SIZE];
static int dma_chan;
static volatile bool using_buffer_a = true;

// Ring buffer functions
static inline void ring_buffer_init(ring_buffer_t *rb) {
    rb->write_idx = 0;
    rb->read_idx = 0;
    rb->count = 0;
}

static inline bool ring_buffer_full(ring_buffer_t *rb) {
    return rb->count == RING_BUFFER_SIZE;
}

static inline bool ring_buffer_empty(ring_buffer_t *rb) {
    return rb->count == 0;
}

static inline void ring_buffer_put(ring_buffer_t *rb, uint16_t value) {
    if (ring_buffer_full(rb)) {
        // Overwrite oldest sample
        rb->read_idx = (rb->read_idx + 1) % RING_BUFFER_SIZE;
    } else {
        rb->count++;
    }
    rb->buffer[rb->write_idx] = value;
    rb->write_idx = (rb->write_idx + 1) % RING_BUFFER_SIZE;
}

static inline uint16_t ring_buffer_get(ring_buffer_t *rb) {
    if (ring_buffer_empty(rb)) {
        return 0;
    }
    uint16_t value = rb->buffer[rb->read_idx];
    rb->read_idx = (rb->read_idx + 1) % RING_BUFFER_SIZE;
    rb->count--;
    return value;
}
```

13

## ece315\_dma\_adc\_x2.c cont'd

- Since there is just one ADC FIFO, the samples from each source are put there in an interleaved fashion
- Must de-interleave them
- The `dma_handler` clears the interrupt request, processes the current buffer, switches the buffer, and starts the DMA again



```
// Process DMA buffer and distribute to ring buffers
static void process_dma_buffer(uint16_t *buffer, size_t length) {
    for (size_t i = 0; i < length; i += NUM_CHANNELS) {
        // ADC samples are interleaved: [CH0, CH1, CH4, CH0, CH1, CH4, ...]
        ring_buffer_put(&adc0_ring, buffer[i]);
        ring_buffer_put(&adc1_ring, buffer[i + 1]);
        ring_buffer_put(&temp_ring, buffer[i + 2]);
    }
}

// DMA interrupt handler
static void dma_handler(void) {
    // Clear the interrupt request
    dma_hw->ints0 = 1u << dma_chan;
    // Process the completed buffer
    if (using_buffer_a) {
        process_dma_buffer(dma_buffer_a, DMA_BUFFER_SIZE);
        // Switch to buffer B for next transfer
        dma_channel_set_write_addr(dma_chan, dma_buffer_b, true);
    } else {
        process_dma_buffer(dma_buffer_b, DMA_BUFFER_SIZE);
        // Switch to buffer A for next transfer
        dma_channel_set_write_addr(dma_chan, dma_buffer_a, true);
    }

    using_buffer_a = !using_buffer_a;
}
```

14

## ece315\_dma\_adc\_x2.c cont'd

- The ADC is set up for three (3) inputs
  - `ain_sel[0]`, GPIO[26] is connected to the thermistor
  - `ain_sel[1]`, GPIO[27] is connected to the potentiometer
  - `ain_sel[4]` is connected to the internal temperature circuit
- Set for round-robin sampling
- Set ADC clock to 3 kHz
- Set up the ADC FIFO for DMA access
  - Don't shift to get 8-bit samples; want all 12 bits, which fit in 16 bits, 2 bytes



```
// Initialize ADC with round-robin for multiple channels
static void init_adc(void) {
    // Initialize ADC hardware
    adc_init();

    // Configure GPIO26 as ADC input (ADC0)
    adc_gpio_init(26);

    // Configure GPIO27 as ADC input (ADC1)
    adc_gpio_init(27);

    // Enable temperature sensor (ADC4)
    adc_set_temp_sensor_enabled(true);

    // Configure ADC for round-robin sampling
    // Channels 0 and 4
    adc_set_round_robin((1 << ADC_CHANNEL_0) |
                       (1 << ADC_CHANNEL_1) |
                       (1 << ADC_CHANNEL_TEMP));

    // Configure ADC clock divider for desired sample rate
    // ADC clock = 48 MHz
    // Round-robin will alternate channels, so effective sample rate is
    // half what the ADC clock is set to. If we set it to 3 kSps, the
    // channel rate will be 1 kSps
    // Clock divider = 48 MHz / 3 kHz = 16000
    adc_set_clkdiv(16000.0f - 1.0f); // -1 because register is (div - 1)

    // Configure FIFO
    adc_fifo_setup(
        true, // Enable FIFO
        true, // Enable DMA data request (DREQ)
        1,    // DREQ threshold (assert when >= 1 sample)
        false, // Disable error bit
        false // Don't shift 8-bit values
    );
}
```

15

## ece315\_dma\_adc\_x2.c cont'd

- Set up the DMA to transfer 16-bits, 2 bytes, at a time
- Set writes to increment by 2 bytes
- Set reads to stay at same address, the ADC FIFO
- Set the DMA requisition interrupt to be triggered by the ADC
- Finally, configure the channel and define the DMA handler function



```
// Initialize DMA for ADC transfer
static void init_dma(void) {
    // Get a free DMA channel
    dma_chan = dma_claim_unused_channel(true);

    // Configure DMA channel
    dma_channel_config cfg = dma_channel_get_default_config(dma_chan);

    // Transfer 16-bit values
    channel_config_set_transfer_data_size(&cfg, DMA_SIZE_16);

    // Increment write address (to fill buffer)
    channel_config_set_write_increment(&cfg, true);

    // Don't increment read address (always read from ADC FIFO)
    channel_config_set_read_increment(&cfg, false);

    // Set DREQ to ADC
    channel_config_set_dreq(&cfg, DREQ_ADC);

    // Chain to self for continuous operation
    channel_config_set_chain_to(&cfg, dma_chan);

    // Apply configuration
    dma_channel_configure(
        dma_chan,
        &cfg,
        dma_buffer_a,           // Write address (initial buffer)
        &adc_hw->fifo,           // Read address (ADC FIFO)
        DMA_BUFFER_SIZE,       // Transfer count
        false,                  // Don't start yet
    );

    // Enable DMA interrupt
    dma_channel_set_irq0_enabled(dma_chan, true);
    irq_set_exclusive_handler(DMA_IRQ_0, dma_handler);
    irq_set_enabled(DMA_IRQ_0, true);
}
```

16

## ece315\_dma\_adc\_x2.c cont'd

- Define some data processing functions
- The function to calculate the average value will take any of the ring buffers we defined



```
// Convert raw ADC value to voltage
static float adc_to_voltage(uint16_t raw) {
    return (raw * 3.3f) / 4096.0f;
}

// Convert raw temperature sensor ADC value to degrees Celsius
static float adc_to_temperature(uint16_t raw) {
    // RP2040 datasheet formula:
    // T = 27 - (ADC_voltage - 0.706) / 0.001721
    float voltage = adc_to_voltage(raw);
    return 27.0f - (voltage - 0.706f) / 0.001721f;
}

// Calculate average of last N samples
static float calculate_average(ring_buffer_t *ring_buf, uint32_t num_samples) {
    if (num_samples > ring_buf->count) {
        num_samples = ring_buf->count;
    }

    if (num_samples == 0) return 0.0f;

    uint32_t sum = 0;
    uint32_t idx = (ring_buf->write_idx - num_samples + RING_BUFFER_SIZE) % RING_BUFFER_SIZE;

    for (uint32_t i = 0; i < num_samples; i++) {
        sum += ring_buf->buffer[idx];
        idx = (idx + 1) % RING_BUFFER_SIZE;
    }

    return (float)sum / num_samples;
}
```

17

## ece315\_dma\_adc\_x2.c cont'd

- Main starts by initializing the ring buffers
- Initialize the ADC and DMA
- Start the ADC, then the DMA



```
int main(void) {
    // Initialize stdio
    stdio_init_all();

    // Wait for USB serial connection (optional, comment out for standalone operation)
    sleep_ms(2000);

    printf("\n\n=== RP2040 Dual-Channel DMA ADC ===\n\n");
    printf("Sample Rate: %d Hz per channel\n", SAMPLE_RATE_HZ / NUM_CHANNELS);
    printf("Ring Buffer Size: %d samples\n", RING_BUFFER_SIZE);
    printf("DMA Buffer Size: %d samples\n", DMA_BUFFER_SIZE);

    // Initialize ring buffers
    ring_buffer_init(&adc0_ring);
    ring_buffer_init(&adc1_ring);
    ring_buffer_init(&temp_ring);

    // Initialize ADC
    init_adc();

    sleep_ms(1000);

    // Initialize DMA
    init_dma();

    printf("Starting acquisition...\n\n");

    // Start ADC free-running mode
    adc_run(true);

    // Start DMA transfer
    dma_channel_start(dma_chan);

    // Main loop - demonstrate data access
    uint32_t last_print_time = 0;

    while (true) {
        uint32_t now = to_ms_since_boot(get_absolute_time());

        // Print statistics every 1 second
        if (now - last_print_time >= 1000) {
            last_print_time = now;

            // Calculate and display running averages
            if (adc0_ring.count >= 100) {
                float avg_adc0 = calculate_average(&adc0_ring, 100);
                float avg_adc1 = calculate_average(&adc1_ring, 100);
                float avg_temp = calculate_average(&temp_ring, 100);
                static ring_buffer_t adc0_ring;

                printf("100-sample averages:\n");
                printf("  ADC0: %.1f counts (%.3f V)\n",
                    avg_adc0, adc_to_voltage((uint16_t)avg_adc0));
                printf("  ADC1: %.1f counts (%.3f V)\n",
                    avg_adc1, adc_to_voltage((uint16_t)avg_adc1));
                printf("  Temp: %.1f counts (%.1f °C)\n\n",
                    avg_temp, adc_to_temperature((uint16_t)avg_temp));
            }

            // Your application logic here

            sleep_ms(10);
        }
    }

    return 0;
}
```

18

## ece315\_dma\_adc\_x2.c cont'd

- Now just let things run
- Periodically calculate the average value of each source



```
// Main loop - demonstrate data access
uint32_t last_print_time = 0;

while (true) {
    uint32_t now = to_ms_since_boot(get_absolute_time());

    // Print statistics every 1 second
    if (now - last_print_time >= 1000) {
        last_print_time = now;

        // Calculate and display running averages
        if (adc0_ring.count >= 100) {
            float avg_adc0 = calculate_average(&adc0_ring, 100);
            float avg_adc1 = calculate_average(&adc1_ring, 100);
            float avg_temp = calculate_average(&temp_ring, 100);
            static ring_buffer_t adc0_ring;

            printf("100-sample averages:\n");
            printf("  ADC0: %.1f counts (%.3f V)\n",
                avg_adc0, adc_to_voltage((uint16_t)avg_adc0));
            printf("  ADC1: %.1f counts (%.3f V)\n",
                avg_adc1, adc_to_voltage((uint16_t)avg_adc1));
            printf("  Temp: %.1f counts (%.1f °C)\n",
                avg_temp, adc_to_temperature((uint16_t)avg_temp));
        }
        // Your application logic here

        sleep_ms(10);
    }
}

return 0;
}
```

19

## ece315\_dma\_adc\_x2.c cont'd

- Now just let things run
- Periodically calculate the average value of each source
- Are we missing any samples when we calculate the averages?
  - Remember, the DMA operates independently of the CPU



```
// Main loop - demonstrate data access
uint32_t last_print_time = 0;

while (true) {
    uint32_t now = to_ms_since_boot(get_absolute_time());

    // Print statistics every 1 second
    if (now - last_print_time >= 1000) {
        last_print_time = now;

        // Calculate and display running averages
        if (adc0_ring.count >= 100) {
            float avg_adc0 = calculate_average(&adc0_ring, 100);
            float avg_adc1 = calculate_average(&adc1_ring, 100);
            float avg_temp = calculate_average(&temp_ring, 100);
            static ring_buffer_t adc0_ring;

            printf("100-sample averages:\n");
            printf("  ADC0: %.1f counts (%.3f V)\n",
                avg_adc0, adc_to_voltage((uint16_t)avg_adc0));
            printf("  ADC1: %.1f counts (%.3f V)\n",
                avg_adc1, adc_to_voltage((uint16_t)avg_adc1));
            printf("  Temp: %.1f counts (%.1f °C)\n\n",
                avg_temp, adc_to_temperature((uint16_t)avg_temp));
        }
        // Your application logic here

        sleep_ms(10);
    }
}

return 0;
}
```

20

## Wokwi sadness

- Unfortunately, we cannot run this example on Wokwi
- DMA only supports the PIOs

(I tried 😞)



### Simulation features

The Raspberry Pi Pico is simulated using the [RP2040js Library](#).

This table summarizes the status of the simulation features:

| Peripheral     | Status | Notes                                                                |
|----------------|--------|----------------------------------------------------------------------|
| Processor core | ✓      | Only a single core is simulated                                      |
| GPIO           | ✓      |                                                                      |
| PIO            | ✓      | PIO Debugger available                                               |
| USB            | ●      | USB CDC (Serial) supported, see <a href="#">Serial Monitor</a> below |
| UART           | ✓      |                                                                      |
| I2C            | ✓      | Master mode only                                                     |
| SPI            | ✓      | Master mode only                                                     |
| PWM            | ✓      |                                                                      |
| DMA            | ✓      | Only for the PIO peripheral                                          |
| Timer          | ✓      | Pausing the timer not implemented yet                                |

21

## Summary

- Many embedded systems must interface with analogue real-world signals (IoT, automotive, HVAC, biomedical, audio).
- DMA enables high-rate, low-overhead data acquisition by moving ADC samples directly into memory without CPU involvement.
- In digitally-controlled analogue systems, ADCs, DMA, DACs, memory buffers, and CPUs can form effective feedback control systems.
- The RP2040 ADC supports multi-channel, round-robin sampling with a shared FIFO, requiring sample de-interleaving.
- Ring buffers and DMA interrupt handlers enable continuous sampling while the CPU performs processing asynchronously.
- Because DMA operates independently of the CPU, careful design is required to maintain data consistency and avoid timing issues.

## Next Lecture

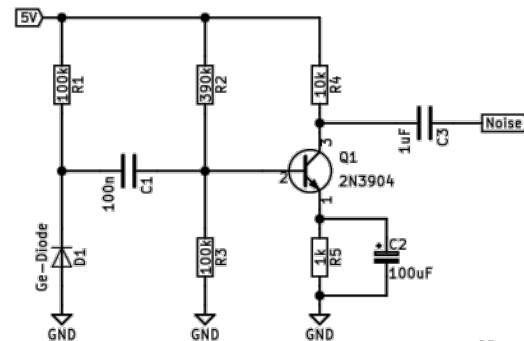
- Interrupts and Events

# Questions



## Noise source

- A reverse-biased diode that will go into breakdown (pass current) is a very good generator of broadband noise
- Circuits like the one shown are used in radio, test instrument, and other applications
- This circuit will produce 1 Vpp white noise over a span of many MHz



<https://noisio.de/blog/small-voltage-noise-source>